CHAPTER 4

Data Correctness and Consistency

Now you're ready to learn how to use Constraints to improve your data layer. PostgreSQL offers a variety of constraint types and options that can help you make sure your data is correct and consistent. These capabilities are up to you to implement.

The general technique you'll follow is to add Constraint objects at various levels such as at the table level or column level. In most Ruby on Rails applications, you'll also validate data at the application layer using the Active Record Validations framework.

You'll compare Database level and Application level constraints and validation approaches to better understand their trade-offs.

To help understand what consistent and valid data looks like, you'll start with examples of invalid and inconsistent data. The first example you'll look at is when data violates *Referential Integrity*. Referential Integrity refers to data in two tables with a referring or referenced relationship, and making sure both sides are intact.

When Referential Integrity is violated it means that one half of the relationship is missing. Referential integrity violations are a common type of application bug.

To begin enforcing referential integrity you'll add Foreign Key constraints. Foreign Key constraints work together with Primary Key columns on another table and this is the mechanism you use to get PostgreSQL to enforce the relationship.

In order to add a validated constraint, there must be no pre-existing violations. You'll see ways to work around this by modifying this behavior. Referential integrity can also be enforced at the application layer in Active Record model code.